Engenharia de Características para Aprendizagem Computacional / Engenharia de Atributos

Application Domain: Signal (part 2)

Licenciatura em Enheraria Informática / Licenciatura em Inteligência Artificial e Ciência de Dados / Licenciatura em Engenharia e Ciência de Dados
Edition 2025-26

Marco Simões

Wavelets (ondoletas)

Wavelets-CWT

Formulação Matemática

- Wavelet

$$\Psi_{a,b}(t) = \frac{1}{\sqrt{a}} \Psi\left(\frac{t-b}{a}\right)$$

- Análise

$$w_{a,b} = \left\langle \Psi_{a,b}^{*}(t) f(t) \right\rangle = \int_{-\infty}^{+\infty} \Psi_{a,b}^{*}(t) f(t) dt$$

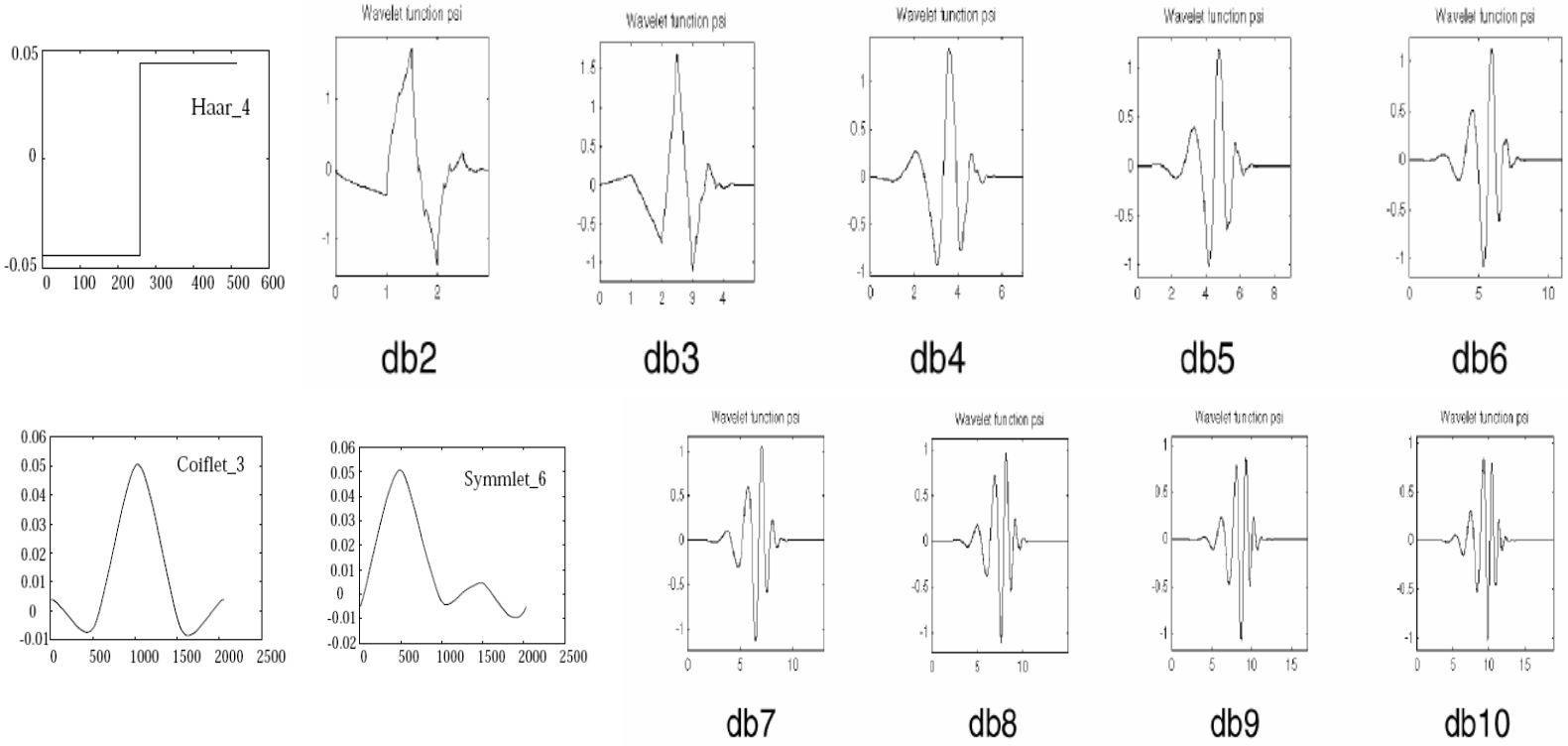
- Síntese

$$f(t) = \frac{1}{C_{\Psi}} \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} w_{a,b} \Psi_{a,b}(t) \frac{dadb}{a^2}$$

$$C_{\Psi} = \int_{0}^{+\infty} \frac{\left|\Psi_{a,b}(w)\right|^2}{w} dw$$

Wavelets-CWT

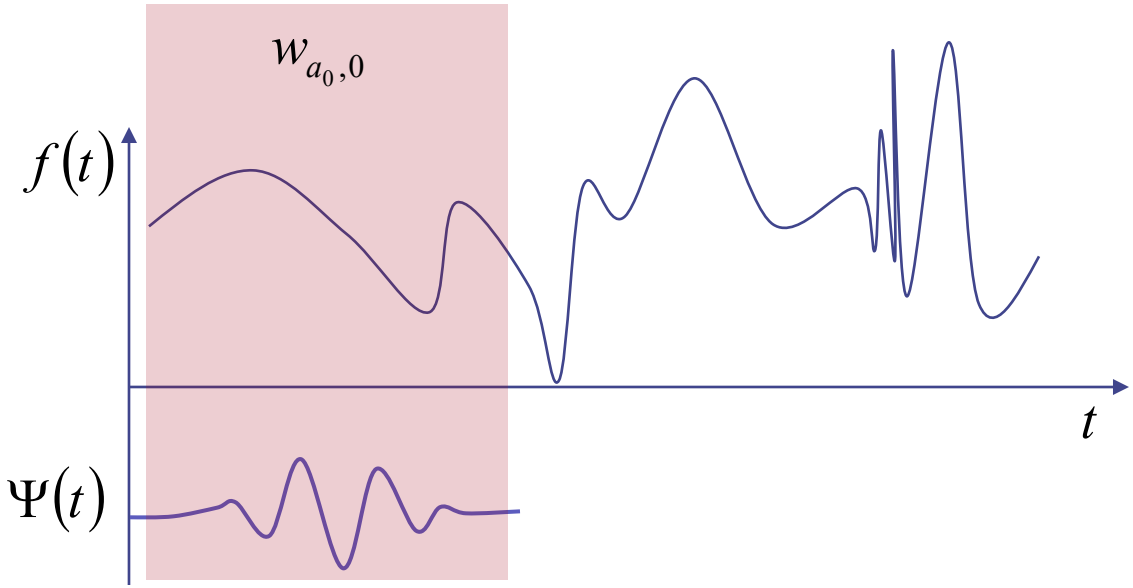
- Mother wavelets
  - Existem muitas mother wavelets definidas



Wavelets-CWT

- Transformada contínua de Wavelet: Como calcular os coeficientes?
- Um processo em 4 passos:
  - Passo 1: Começar com uma wavelet e compará-la com o sinal.

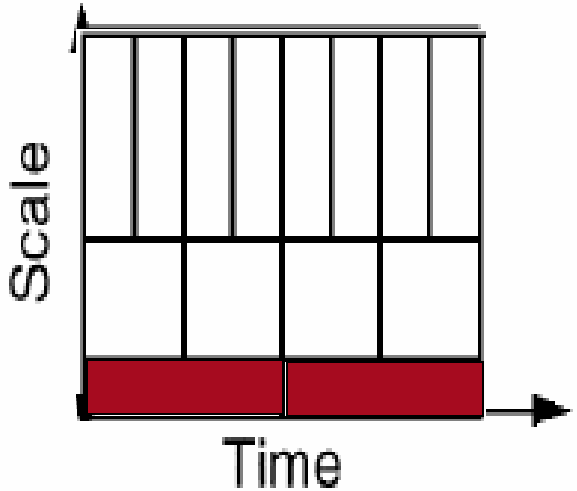
$$w_{a_0,0} = \int_{-\infty}^{+\infty} \Psi_{a_0,0}(t) f(t) dt = \int_{k_2}^{k_1} \Psi_{a_0,0}(t) f(t) dt$$

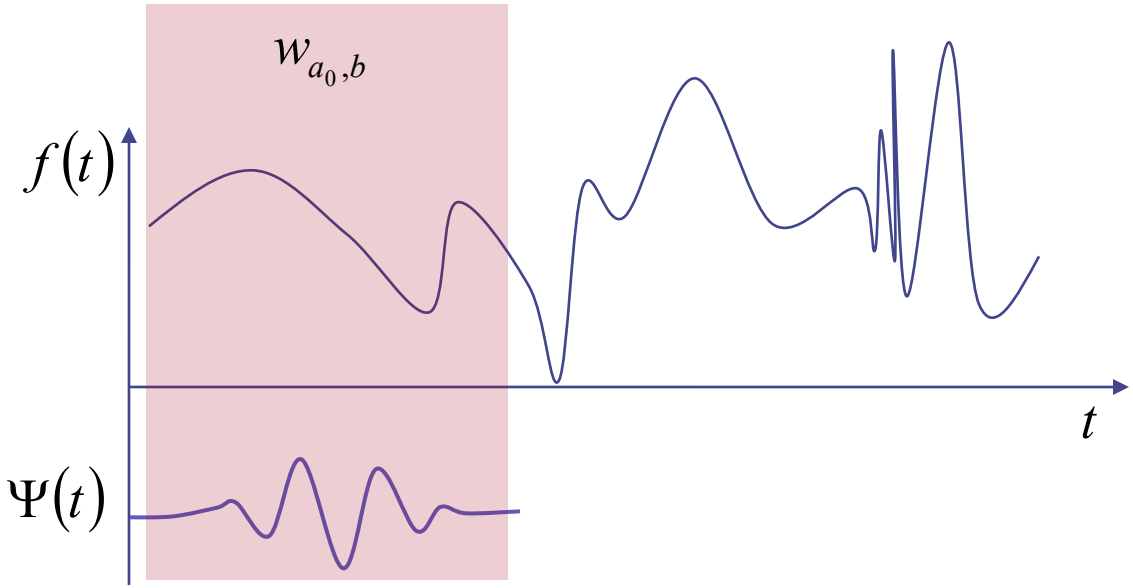


Wavelets-CWT

- Transformada contínua de Wavelet: Como calcular os coeficientes?
  - Passo 2: Transladar a wavelet e repetir o passo 1.

$$w_{a_0,b} = \int_{-\infty}^{+\infty} \Psi_{a_0,b}(t) f(t) dt = \int_{k_2}^{k_1} \Psi_{a_0,b}(t) f(t) dt$$

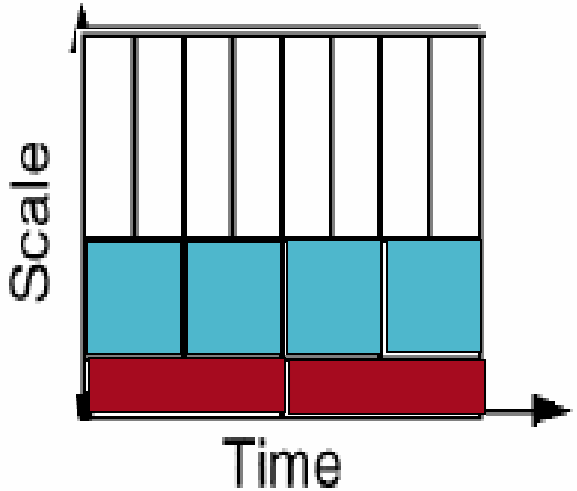


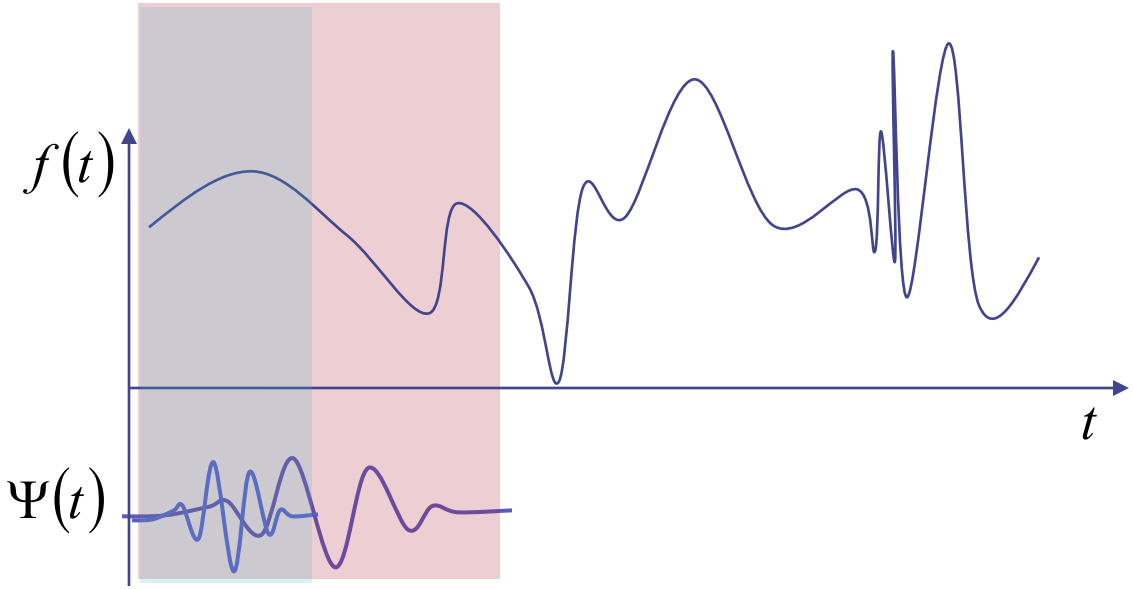


Wavelets-CWT

- Transformada contínua de Wavelet: Como calcular os coeficientes?
  - Passo 3: Alterar a escala da wavelet e repetir os passos 1 e 2.

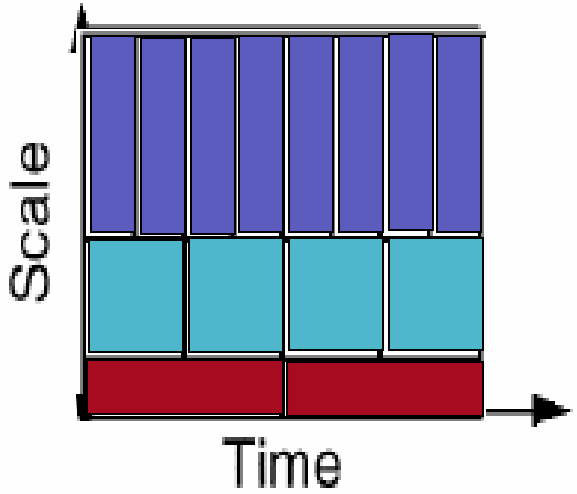
$$w_{a_1,b} = \int_{-\infty}^{+\infty} \Psi_{a_1,b}(t) f(t) dt = \int_{k_2}^{k_1} \Psi_{a_1,b}(t) f(t) dt$$

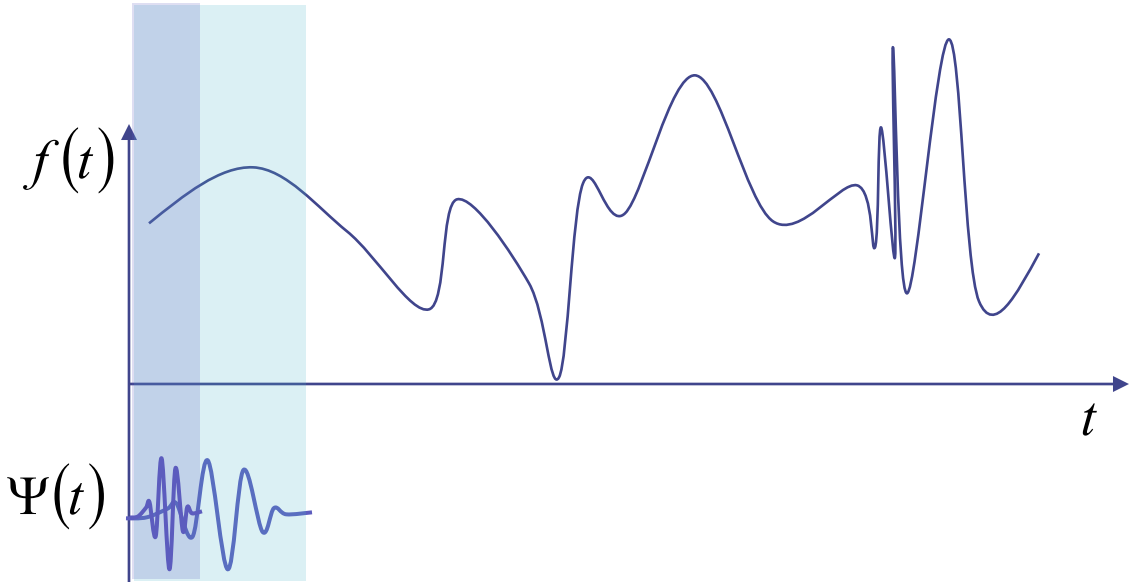




Wavelets-CWT

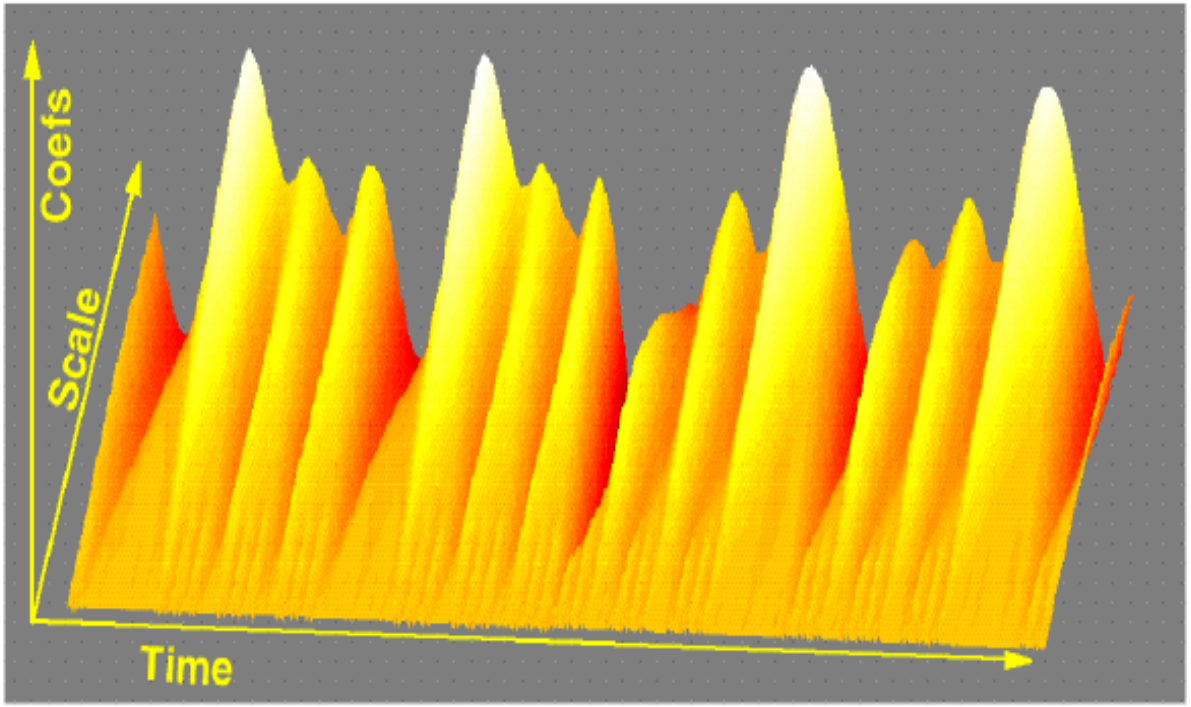
- Transformada contínua de Wavelet: Como calcular os coeficientes?
  - Passo 4: Repetir passo 3 para todas as escalas.





Wavelets-CWT

- Resultado



Wavelets-CWT

- Interpretação (escala e frequência)
  - Escala alta (Wavelet comprimida)
    - Detalhes finos → frequências altas
  - Escala baixa (Wavelet dilatada)
    - Detalhes menos finos → frequências baixas

Wavelets-CWT

- Tipos de transformadas
  - Transformada contínua (CWT)
    - Problemas:
      - Demasiado complexo (regra geral não tem solução analítica)
      - Como determinar as gamas de variação de *a* e de *b*?
      - Demasiados dados!

  - Transformadas discretas
    - Função contínua, parâmetros discretos (*a* e *b*)
      - Variações discretas de *a* e de *b*

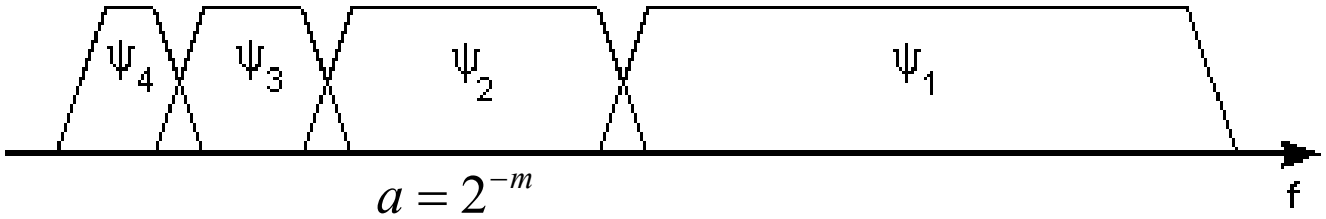
    - Função discreta (amostrada), parâmetros discretos (*a* e *b*)
      - A situação mais comum em aplicações digitais.

Wavelets - DWT

- Transformadas discretas
  - Análise em frequência

$$F\{\Psi(at)\} = \frac{1}{|a|}\Psi\left(\frac{w}{a}\right)$$

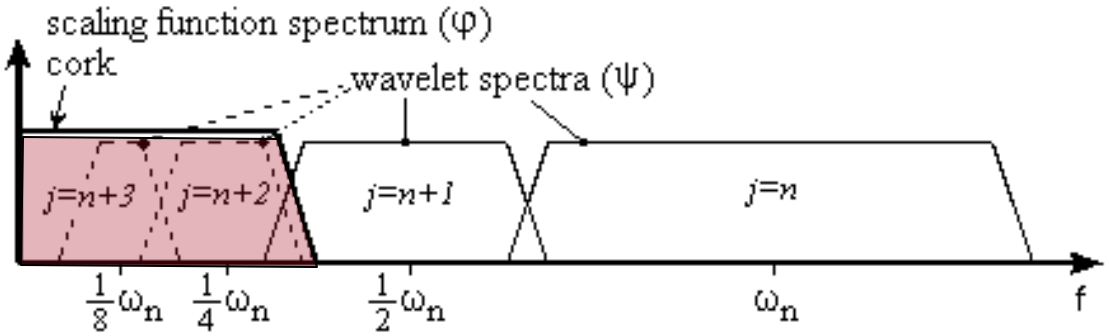
$$f(t) = \sum_m \sum_n w_{m,n}\,\Psi\left(a_0^m t - nb_0\right)$$



  - $a = 2^{-m}$
  - Em cada nível reduzimos em 2 a banda passante e a frequência central.
  - Para poder reconstruir o espectro é necessário cobrir a gama de frequências que compõem o sinal completamente.
    - Bandas têm que se sobrepor parcialmente! (Basta escolher convenientemente a mother wavelet)
    - Para cobrir as frequências até 0 → número infinito de escalas!

Wavelets

- Transformadas discretas
  - Problema: Número infinito de escalas para cobrir toda a zona.
  - Solução: vamos guardar essa parte com informação adicional!

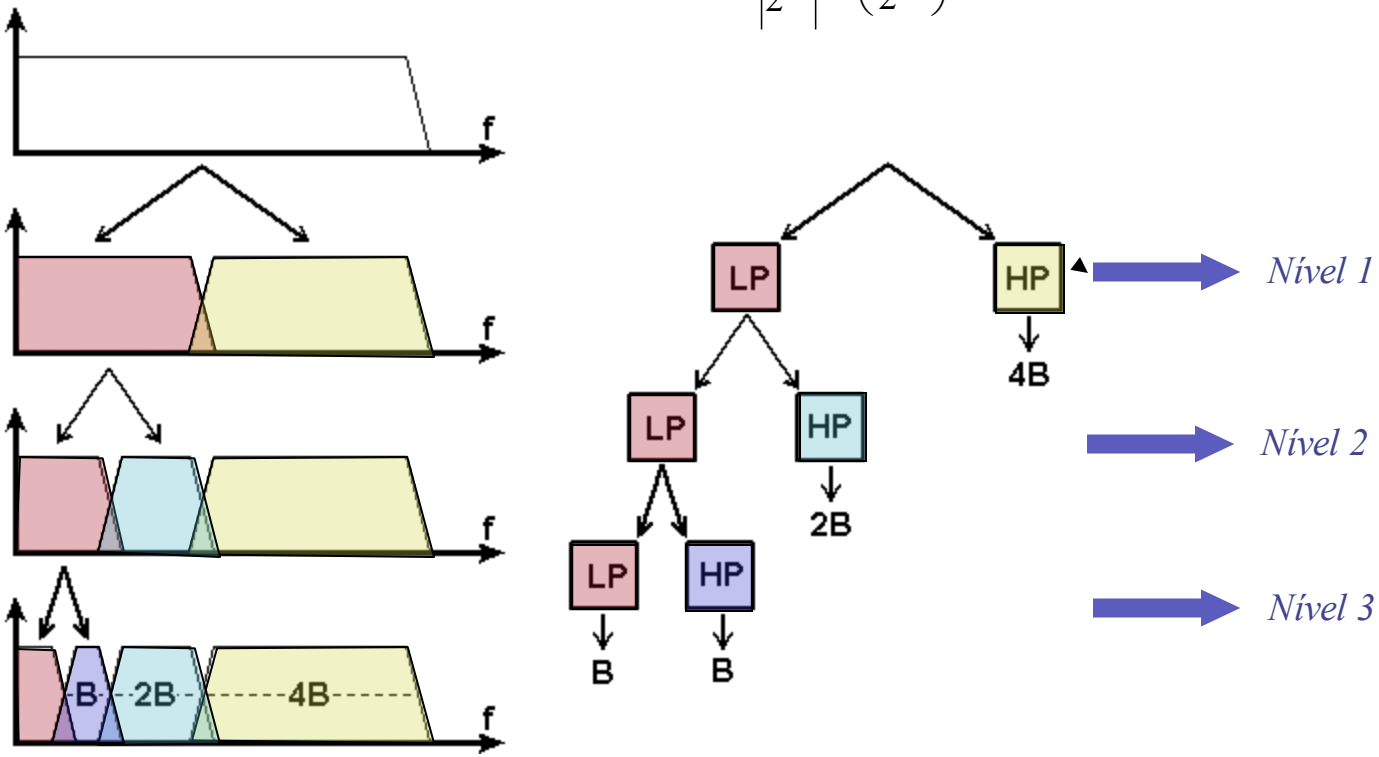


  - A parte que falta é de baixas frequências, pelo que poderá ser obtida por um filtro passa-baixo – a função de escala!

Wavelets

- Transformadas discretas
  - Abordagem Geral

$$F\{\Psi(2^{-j}t)\} = \frac{1}{|2^{-j}|}\Psi\left(\frac{w}{2^{-j}}\right) = 2^{j}\Psi(2^{j}w)$$

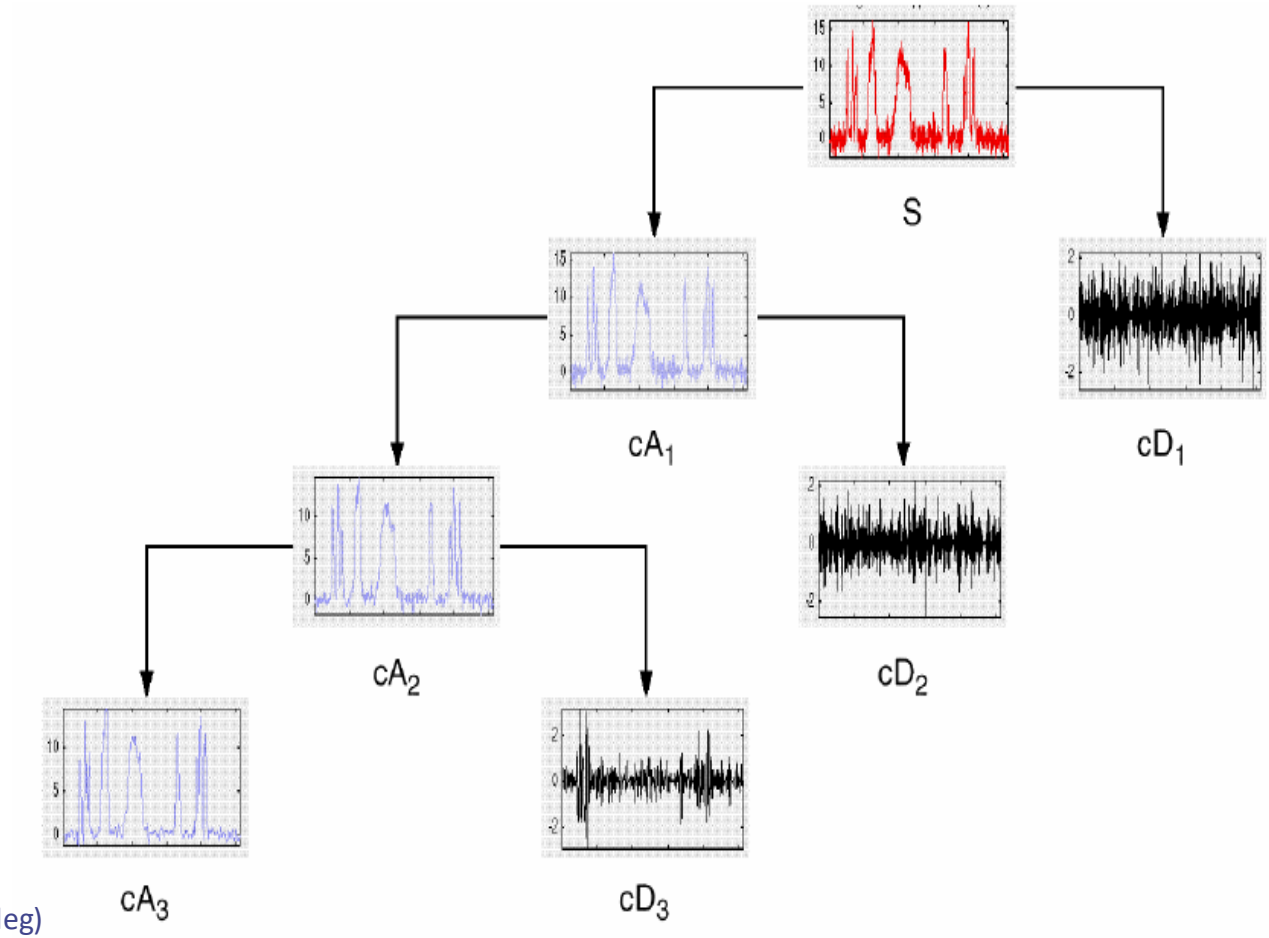


Wavelets

- Transformadas discretas
  - **Coeficientes de Aproximação**
    - É a parte mais importante do sinal na generalidade das aplicações.
    - Corresponde às componentes de baixa frequência do sinal.
  - **Coeficientes de Detalhe**
    - É a parte do sinal correspondente aos pormenores (eventualmente ruído)
    - Corresponde à componente de alta frequência do sinal.

Wavelets

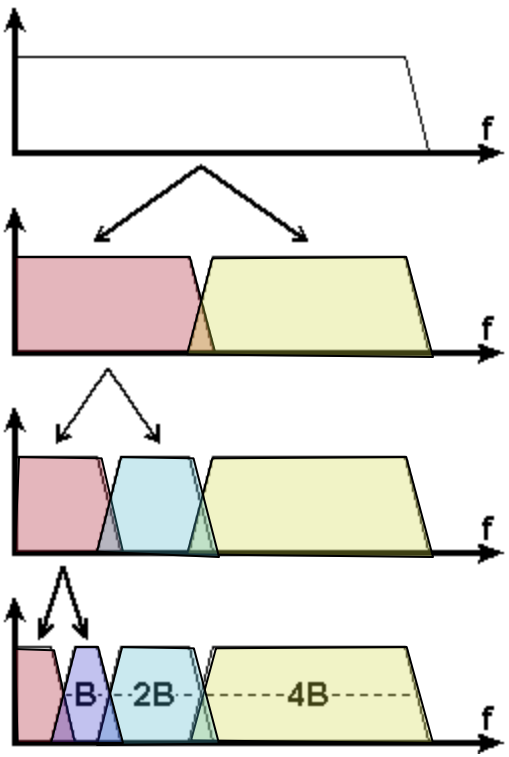
- Exemplo

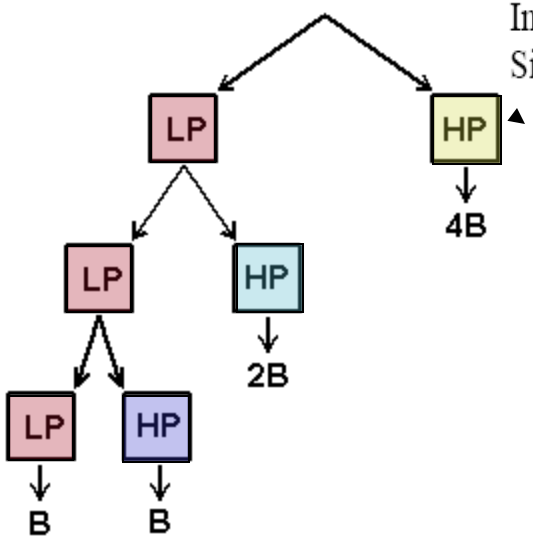


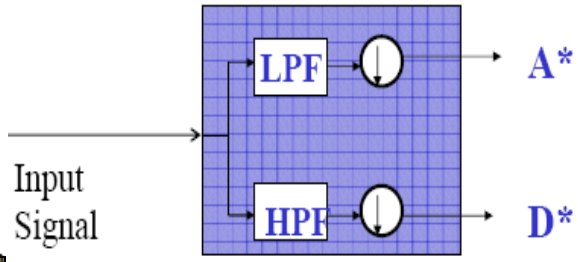
(retirado de Nimrod Peleg)

Wavelets

- DWT







Wavelets

- Quantos níveis?
  - Teoricamente até só existir um coeficiente de aproximação!
  - Na prática terminámos antes! Depende da natureza do sinal ou de algum critério (exemplo: entropia)

CWT em Python – hands on!

- Explore pyWavelets:
  - Explore wavelet families
  - See Continuous vs Discrete wavelets

Built-in wavelets - `wavelist()`

`pywt.wavelist(family=None, kind='all')`

Returns list of available wavelet names for the given family name.

Parameters: **family** : *str, optional*

Short family name. If the family name is None (default) then names of all the built-in wavelets are returned. Otherwise the function returns names of wavelets that belong to the given family. Valid names are:

```
'haar', 'db', 'sym', 'coif', 'bior', 'rbio', 'dmey', 'gaus',
'mexh', 'morl', 'cgau', 'shan', 'fbsp', 'cmor'
```

kind : *{'all', 'continuous', 'discrete'}, optional*

Whether to return only wavelet names of discrete or continuous wavelets, or all wavelets. Default is `'all'`. Ignored if `family` is specified.

Returns: **wavelist** : *list of str*

List of available wavelet names.

Examples

```
>>> import pywt
>>> pywt.wavelist('coif')
['coif1', 'coif2', 'coif3', 'coif4', 'coif5', 'coif6', 'coif7', ...
>>> pywt.wavelist(kind='continuous')
['cgau1', 'cgau2', 'cgau3', 'cgau4', 'cgau5', 'cgau6', 'cgau7', ...
```

Wavelet `families()`

`pywt.families(short=True)`

Returns a list of available built-in wavelet families.

Currently the built-in families are:

- Haar (`haar`)
- Daubechies (`db`)
- Symlets (`sym`)
- Coiflets (`coif`)
- Biorthogonal (`bior`)
- Reverse biorthogonal (`rbio`)
- *"Discrete"* FIR approximation of Meyer wavelet (`dmey`)
- Gaussian wavelets (`gaus`)
- Mexican hat wavelet (`mexh`)
- Morlet wavelet (`morl`)
- Complex Gaussian wavelets (`cgau`)
- Shannon wavelets (`shan`)
- Frequency B-Spline wavelets (`fbsp`)
- Complex Morlet wavelets (`cmor`)

Parameters: **short** : *bool, optional*

Use short names (default: True).

Returns: **families** : *list*

List of available wavelet families.

Examples

```
>>> import pywt
>>> pywt.families()
['haar', 'db', 'sym', 'coif', 'bior', 'rbio', 'dmey', 'gaus', 'mexh', 'morl', 'cgau', 'shan', 'fbs
>>> pywt.families(short=False)
['Haar', 'Daubechies', 'Symlets', 'Coiflets', 'Biorthogonal', 'Reverse biorthogonal', 'Discrete Me
```

CWT em Python – hands on!

- Explore pyWavelets:
  - Create a scalogram with CWT

Continuous Wavelet Transform (CWT)

This section describes functions used to perform single continuous wavelet transforms.

Single level - `cwt`

`pywt.cwt(data, scales, wavelet)`

One dimensional Continuous Wavelet Transform.

Parameters:

data : *array_like*

Input signal

scales : *array_like*

The wavelet scales to use. One can use `f = scale2frequency(wavelet, scale)/sampling_period` to determine what physical frequency, `f`. Here, `f` is in hertz when the `sampling_period` is given in seconds.

wavelet : *Wavelet object or name*

Wavelet to use

sampling_period : *float*

Sampling period for the frequencies output (optional). The values computed for `coefs` are independent of the choice of `sampling_period` (i.e. `scales` is not scaled by the sampling period).

method : *{'conv', 'fft'}, optional*

The method used to compute the CWT. Can be any of:

- `conv` uses `numpy.convolve`.
- `fft` uses frequency domain convolution.
- `auto` uses automatic selection based on an estimate of the computational complexity at each scale.

The `conv` method complexity is `O(len(scale) * len(data))`. The `fft` method is `O(N * log2(N))` with `N = len(scale) + len(data) - 1`. It is well suited for large size signals but slightly slower than `conv` on small ones.

axis: int, optional

Axis over which to compute the CWT. If not given, the last axis is used.

Returns:

coefs : *array_like*

Continuous wavelet transform of the input signal for the given scales and wavelet. The first axis of `coefs` corresponds to the scales. The remaining axes match the shape of `data`.

frequencies : *array_like*

If the unit of sampling period are seconds and given, than frequencies are in hertz. Otherwise, a sampling period of 1 is assumed.

DWT em Python – hands on!

- Explore pyWavelets:
  - Perform a DWT

```
pywt.wavedec(data, wavelet, mode='symmetric', level=None, axis=-1)
```

Multilevel 1D Discrete Wavelet Transform of data.

Parameters:

data: array_like

Input data

wavelet : ***Wavelet object or name string***

Wavelet to use

mode : ***str, optional***

Signal extension mode, see Modes.

level : ***int, optional***

Decomposition level (must be >= 0). If level is None (default) then it will be calculated using the `dwt_max_level` function.

axis: int, optional

Axis over which to compute the DWT. If not given, the last axis is used.

Returns:

[cA_n, cD_n, cD_n-1, …, cD2, cD1] : ***list***

Ordered list of coefficients arrays where `n` denotes the level of decomposition. The first element (`cA_n`) of the result is approximation coefficients array and the following elements (`cD_n` - `cD_1`) are details coefficients arrays.